

Software Requirements Specification (SRS)

GPA and WGPA Calculation Web Application

1. Introduction

1.1 Purpose This document outlines the software requirements for a web-based GPA and WGPA calculation application tailored for university students. The application will allow students to calculate their Semester Grade Point Average (GPA) and final Weighted Grade Point Average (WGPA) based on their chosen specialization, academic year, and selected modules (including electives). It also provides a printable summary of their academic performance.

1.2 Technology Stack Strategy (SEO Compatibility) To meet the requirement for Search Engine Optimization (SEO) compatibility, the application will be built using **Next.js** (a React framework). Next.js provides Server-Side Rendering (SSR) and Static Site Generation (SSG), ensuring that search engine crawlers can index the page content, meta tags, and structural data effectively, which is critical for organic discoverability.

- **Frontend Framework:** Next.js (React)
- **Styling:** Tailwind CSS (for rapid, mobile-first responsive design)
- **State Management:** React Context API or Zustand (for managing grades across semesters)

2. Functional Requirements

2.1 Specialization Selection

- The system shall provide a dropdown interface for students to select their degree specialization.
- Defined specializations include: Software Engineering (SE), Information Technology (IT), Data Science (DS), Interactive Media (IM), Computer Systems & Network Engineering (CSNE), and Cyber Security.

2.2 Semester and Module Management

- Students shall be able to view and interact with modules categorized by Academic Year and Semester (e.g., Year 1 Semester 1 through Year 4 Semester 2).
- Upon selecting a semester, the system shall dynamically populate the corresponding mock modules.
- **Elective Module Handling:** For 4th-year semesters, the system must provide a dropdown or selection mechanism allowing the student to choose 1 or 2 elective modules from a predefined pool of available electives to add to their semester calculation.
- Each module row shall include a dropdown menu for the student to select their achieved grade (A+, A, A-, etc.).

2.3 GPA Calculation

- The system shall calculate the Semester GPA based on the standard formula: $\text{Semester GPA} = \frac{\text{Total Grade Points}}{\text{Total Credits}}$ (Where $\text{Grade Points} = \text{Module Credit Value} \times \text{Grade Value}$)
- The GPA shall update dynamically as the user inputs or changes their grades.

2.4 WGPA Calculation

- The system shall compute the overall Weighted Grade Point Average (WGPA) based on the specific academic year weightings:
 - Year 1: 0% (Credits count toward graduation, but grades do not impact final WGPA)
 - Year 2: 20%
 - Year 3: 30%
 - Year 4: 50%
- The WGPA shall be displayed prominently and update in real-time as users enter grades across the different academic years.

2.5 Report Generation & Printing

- The application shall include a "Print Results" feature.
- Activating this feature shall trigger a browser print dialog with a specially formatted, print-friendly layout (CSS @media print).
- The print layout shall hide interactive elements (dropdowns, navigation bars) and clearly display the student's Specialization, a clean table of Modules and Grades, Semester GPAs, and the final WGPA.

3. Grading System Specifications

The application's calculation engine must strictly adhere to the following 12-grade system to assess student performance. A student must score a grade of 'C' or above to obtain a pass in a subject.

Grade	Grade Points	Marks Range
A+	4.00	90 - 100
A	4.00	80 - 89
A-	3.70	75 - 79
B+	3.30	70 - 74
B	3.00	65 - 69
B-	2.70	60 - 64
C+	2.30	55 - 59
C	2.00	45 - 54
C-	1.70	40 - 44
D+	1.30	35 - 39
D	1.00	30 - 34
E	0.00	00 - 29

4. Non-Functional Requirements

- **Responsive Design:** The application must follow a mobile-first design approach. Data tables and grading interfaces must scale down gracefully for smartphones (e.g., using card layouts or scrollable overflow tables on smaller viewports).
- **Usability:** The interface must be intuitive. Users should be able to navigate between fields using the "Tab" key for rapid data entry.
- **Performance:** Real-time calculations must happen instantaneously without noticeable lag as the user selects grades.
- **Data Persistence (Optional/Future):** While not explicitly requested for V1, utilizing `localStorage` to save the user's entered grades across browser sessions would enhance the user experience.

5. Mock Data Structure (Implementation Guide)

The following JSON/JavaScript object structure is recommended for seeding the application with mock data. This structure handles standard modules and differentiates them from the 4th-year elective pools.

```
export const curriculumData = {
  "SE": { // Software Engineering Specialization
    "Y1S1": [
      { id: "IT1010", name: "Introduction to Programming", credits: 4, type: "c" },
      { id: "IT1020", name: "Mathematics for Computing", credits: 3, type: "com" },
    ],
    "Y2S1": [
      { id: "SE2010", name: "Object Oriented Programming", credits: 4, type: "c" },
      { id: "SE2020", name: "Database Management Systems", credits: 4, type: "c" },
    ],
    "Y3S1": [
      { id: "SE3010", name: "Software Architecture", credits: 4, type: "compuls" },
      { id: "SE3020", name: "Web Technologies", credits: 3, type: "compulsory" },
    ],
    "Y4S1": {
      "compulsory": [
        { id: "SE4010", name: "Software Engineering Project I", credits: 4, type: "compulsory" },
      ],
      "electivesPool": [
        { id: "SE4020", name: "Machine Learning Concepts", credits: 3, type: "elective" },
        { id: "SE4030", name: "Cloud Computing", credits: 3, type: "elective" },
        { id: "SE4040", name: "Internet of Things", credits: 3, type: "elective" },
      ],
      "requiredElectivesCount": 1 // Configuration for how many they need to pick
    }
  }
}
// Add IT, DS, IM, CSNE, Cyber Security objects here following the same pattern
};
```